

Symmetry-broken SAT self-certifies the plane’s $\chi \geq 5$ lower bound (and where it stops)

Owen P. Kent*

June 2026

Abstract

The lower bound $\chi(\mathbb{R}^2) \geq 5$ on the chromatic number of the plane rests on certifying that specific finite unit-distance graphs (UDGs) are not 4-colorable. These certifications are k -colorability SAT instances whose naive one-hot encoding carries the full $k!$ color-permutation symmetry, and several of them have been reported as practically intractable. We report a simple but decisive engineering result: exporting a sound symmetry break (clique-fixing plus first-appearance color precedence) as a CNF preamble lets an off-the-shelf CDCL solver decide all of them, each with an optional machine-checkable DRAT proof. Concretely, $M^4(C_5)$ at $k = 6$ (UNSAT) is intractable under the naive encoding but a 22 s solve once broken; the 510-vertex Polymath16 graph at $k = 4$ (UNSAT) drops from ~ 110 s to 1.66 s; and de Grey’s 1585-vertex graph at $k = 4$ (UNSAT), the actual $\chi(\mathbb{R}^2) \geq 5$ certificate, from ~ 92 min to 18 min. The entire known $\chi(\mathbb{R}^2) \geq 5$ computational program is thereby *self-certifiable on a single workstation*. We pair this with two methodological findings. First, a *portfolio law*: near the colorability phase boundary, single-solver runtime is heavy-tailed and misleading; on one identical in-class instance CaDiCaL did not finish in 12 hours while MapleChrono returned SAT in 155 seconds (a factor exceeding 270). Second, an honest *regime boundary*: the symmetry break crushes *structured* UNSAT but does *not* tame *phase-transition* hardness, so it certifies everything currently known and does *not* shortcut the search for a new $\chi \geq 6$ object. The symmetry break itself is standard; the contribution is wiring it into the Hadwiger-Nelson pipeline, measuring that it crosses the specific walls, and mapping where it stops. Finally we record a Σ_2 collapse: for the two-quantifier question “does a given graph class contain a $\chi \geq k$ member on n vertices?”, moving the universal half inside the solver as a chromatic-number propagator decides it without enumerating the class, answering at order 17 in 0.83 cpu-hours what enumeration measured at more than 80 cpu-days. It buys speed at the cost of checkability, and it stops at order 18.

1 Introduction

The Hadwiger-Nelson problem asks for the chromatic number $\chi(\mathbb{R}^2)$ of the unit-distance graph on the Euclidean plane. The classical bounds $5 \leq \chi(\mathbb{R}^2) \leq 7$ have stood for decades; the lower bound was raised from 4 to 5 only recently, by de Grey [4], who exhibited a finite unit-distance graph (UDG) that is not 4-colorable, verified by SAT [5]. (De Grey’s paper reports 1581 vertices; the graph as distributed and certified in this note has 1585.) The Polymath16 project [10] and Parts [9] subsequently reduced the order of such graphs to roughly 510 vertices. Every step of this lower-bound program reduces to the same computational question:

Given a finite graph G and an integer k , is G k -colorable?

*Independent researcher. Correspondence: owenpkent@gmail.com.

A $\chi \geq 5$ certificate is a 4-UNSAT verdict on a UDG; a reduction step is a 5-SAT witness on a smaller graph. The whole edifice is a stack of colorability decisions.

These decisions are routinely handed to a black-box CDCL SAT solver through the standard one-hot encoding (one Boolean $x_{v,c}$ per vertex-color pair). That encoding has a structural defect for graph coloring: it carries the full color-permutation symmetry, so every proper coloring appears $k!$ times in the search space. A solver must either rediscover the symmetry or waste exponential effort exploring symmetric subtrees. In practice this makes several of the program’s own instances slow or intractable: $M^4(C_5)$ at $k = 6$ (a 95-vertex triangle-free Mycielskian whose non-6-colorability certifies $\chi(M^4(C_5)) \geq 7$) was never completed by the naive route within budget, and even routine certificates pay a heavy tax (de Grey’s graph at $k = 4$ takes ~ 92 minutes naively).

This note records that the obstruction is an encoding artifact, not a genuine hardness, and that removing it is cheap. We describe a structure-first colorability solver built for the program (Section 3); the sound symmetry break it searches natively, exported as a CNF preamble for a production CDCL engine (Section 4); the resulting *self-certification* of the entire $\chi(\mathbb{R}^2) \geq 5$ program on one machine (Section 5); and two methodological findings that the program paid for in compute and that we believe transfer to any UDG-colorability effort: a portfolio law and a structured-versus-phase-transition regime boundary (Section 6).

Contributions and honest scope. (1) A from-scratch structure-first colorability solver, validated against a production SAT backend with zero verdict disagreements, that additionally counts colorings *up to color permutation* (a quantity SAT solvers do not expose). (2) A symmetry-broken CNF export that, on the program’s own instances, crosses two specific walls ($M^4(C_5) k=6$ and $P_{510} k=4$) and makes the de Grey lower bound self-certifiable in 18.1 minutes, each with an optional DRAT proof. (3) A portfolio law and a regime boundary, stated as measured facts. (4) A Σ_2 collapse: moving the universal quantifier inside the solver as a propagator decides “does this class contain a $\chi \geq k$ member on n vertices?” without enumerating the class, crossing a wall this program had measured at more than 80 cpu-days. We claim **no** new bound on $\chi(\mathbb{R}^2)$ and **no** novelty in the symmetry break itself, which is the standard lex-leader / clique-fixing construction [3, 12]. The contribution is its application, the measurement that it crosses these particular walls, and the honest delineation of where it does not help. Everything here is reproducible on a single workstation with open-source solvers.

2 Background

The one-hot encoding and its symmetry. The standard CNF encoding of k -colorability introduces a Boolean variable $x_{v,c}$ for each vertex $v \in V(G)$ and color $c \in \{1, \dots, k\}$, with an at-least-one clause $\bigvee_c x_{v,c}$ per vertex and an edge clause $\overline{x_{u,c}} \vee \overline{x_{v,c}}$ for each edge uv and color c (at-most-one clauses per vertex are optional). Any permutation $\sigma \in S_k$ of the colors maps a satisfying assignment to another satisfying assignment; the $k!$ images are distinct points of the search space encoding the same coloring. For UNSAT instances this multiplies the proof effort; for SAT instances it scatters witnesses across symmetric frames.

Breaking the symmetry. A standard remedy is a symmetry-breaking predicate that keeps exactly one representative per orbit [3, 12]. For color symmetry two ingredients suffice and are sound (every proper coloring retains exactly one representative):

- **Clique fixing.** A clique $Q = \{q_1, \dots, q_\omega\}$ must receive ω distinct colors; pin q_i to color i by unit clauses $x_{q_i,i}$. If $\omega > k$ this is an immediate UNSAT certificate.

- **First-appearance precedence.** Order the vertices $v_1, v_2, \dots$ (clique first, then degree-descending) and require that color c may first appear only after color $c - 1$ has appeared: $x_{v_i, c} \rightarrow \bigvee_{j < i} x_{v_j, c-1}$, with rank-bound unit clauses forbidding v_i from using colors above i . This is the lex-leader break specialized to color values.

The two together collapse the $k!$ relabelings to a single canonical coloring.

Checkable certificates. A modern CDCL solver can emit a DRAT proof [13] of an UNSAT verdict, independently checkable by a trusted tool. A symmetry-broken UNSAT proof certifies that the *symmetry-broken* CNF is unsatisfiable; since the added clauses are sound (an orbit representative always survives), this entails that the original graph is not k -colorable.

3 A structure-first colorability solver

We first built a colorability decision procedure from scratch, in pure Python, specialized to the program’s graphs (triangle-rich UDGs, Mycielskians, and near-critical graphs). It is not a general SAT solver and is not meant to compete with CDCL on industrial instances; it exists to make the program’s structure explicit and to serve as the source of the symmetry break exported in Section 4.

Design. Four ingredients, all from scratch with no SAT backend. (i) *Color-class symmetry breaking*: a vertex may introduce color c only if $c \leq 1 + (\text{max color used so far})$, baking the canonical frame of Section 2 directly into the search. (ii) *Clique seeding*: a clique is colored $1.. \omega$ up front, giving an instant $\omega > k$ UNSAT certificate. (iii) *MRV branching with forward checking*: branch on the uncolored vertex with the fewest available colors, backtracking immediately on a domain wipeout. (iv) *Pattern counting*: the number of proper k -colorings *up to color permutation*, which SAT solvers do not provide and which directly measures the “liquidity” of a graph’s coloring space. A node budget bounds every search so the solver never hangs.

Correctness by cross-validation. The solver is validated, not trusted. Across 84 instance/ k pairs (a 60-graph random self-test plus a 24-instance benchmark) its verdict agrees with a production SAT backend (MapleChrono via PySAT [6]) on *every* instance: zero disagreements. Every returned coloring is checked proper, and the pattern counter matches brute-force orbit counts on small graphs (e.g. C_5 has exactly 5 three-coloring patterns).

The ceiling. The solver has no clause learning, so on hard UNSAT instances the search tree explodes: 882 nodes at $M^2(C_5)$ but 447,720 at $M^3(C_5)$, and $M^4(C_5) k=6$ and $P_{510} k=4$ are out of reach in pure Python within budget. Adding conflict-directed backjumping [11] is a large free win on hard structured instances (on $M^3(C_5) k=5$ UNSAT, 13.3 s $\rightarrow$ 1.2 s, and a further $5\times$ under PyPy, $\sim 78\times$ total), but nogood learning without watched-literal propagation does not pay off in pure Python, and $M^4(C_5)$ remains node-count-bound. The lesson is decisive: rather than grow a pure-Python learning engine, *export* the symmetry break to a solver that already learns at full UIP quality.

4 Exporting the symmetry break to CDCL

The export emits the clique-fixing and first-appearance-precedence clauses of Section 2 as a preamble to the one-hot CNF, then hands the result to a production CDCL solver (CaDiCaL [2], Glucose [1],

or MapleChrono, through PySAT [6]). The solver inherits the color-symmetry break it otherwise lacks, on top of its own learning, watched literals, VSIDS, and restarts. A refinement pins an *exact* maximum clique on the high-degree core (cheap because the program’s graphs are sparse, K_4 -free, or triangle-free) for a stronger immediate fix, falling back to a greedy clique.

Soundness and validation. The added clauses are an orbit-selecting predicate, so the symmetry-broken CNF is satisfiable iff G is k -colorable. We validated this empirically as well: over 1000+ instances the symmetry-broken and naive encodings returned identical verdicts with zero bad witnesses (equisatisfiability check).

5 Self-certifying the $\chi(\mathbb{R}^2) \geq 5$ program

The export turns the program’s hardest colorability decisions from walls into seconds-to-minutes solves. All timings below are single isolated runs of CaDiCaL [2] on one workstation; “wall” marks an instance that did not finish within the budget under the naive encoding.

Table 1: Effect of the symmetry-broken export. The naive one-hot encoding does not finish on the $M^4(C_5)$ row within a ~ 120 s conflict budget (a genuine wall, newly crossed); on the other rows it finishes but the break gives one to three orders of magnitude.

Instance	Naive one-hot	Symmetry-broken	Note
$M^3(C_5)$ $k=5$ UNSAT	7.0 s	0.05 s	$\sim 140\times$
$M^4(C_5)$ $k=6$ UNSAT	wall	21.9 s	newly decided ($\chi(M^4(C_5)) \geq 7$)
P_{510} $k=4$ UNSAT	~ 110 s	1.66 s	$\sim 66\times$ (independently re-certified $\chi = 5$)
P_{510} $k=5$ SAT	—	instant	5-coloring witness

The whole lower-bound program, on one machine. With the export, every certificate the $\chi(\mathbb{R}^2) \geq 5$ program depends on is reproducible on a single workstation (Table 2). In particular, de Grey’s 1585-vertex graph is independently confirmed not 4-colorable in 18 minutes (against ~ 92 minutes for the naive encoding), a from-the-graph re-derivation of $\chi(\mathbb{R}^2) \geq 5$, and all nine $\chi = 5$ lineage graphs are certified in seconds each.

Table 2: Self-certification battery (symmetry-broken encoding, CaDiCaL).

Certificate	Time
de Grey 1585 $k=4$ UNSAT ($\chi(\mathbb{R}^2) \geq 5$)	18.1 min
nine lineage graphs (510, 517, 529, 553, 610, 633, 803, 826, 874) $k=4$ UNSAT	1.5–9 s each
$M^4(C_5)$ $k=6$ UNSAT ($\chi(M^4(C_5)) \geq 7$)	22 s

Machine-checkable proofs. Each symmetry-broken UNSAT solve optionally emits a DRAT proof [13]. The break also shrinks the proofs dramatically: $M^3(C_5)$ $k=5$ UNSAT is 13,263 DRAT lines with the break versus 879,467 without (a $\sim 66\times$ reduction), and $M^4(C_5)$ $k=6$ UNSAT yields a complete 1,944,586-line proof in 26 s (the 4 s over the 22 s solve of Table 2 is the proof emission). We stress the honest reading: the proof certifies that the *symmetry-broken* CNF is unsatisfiable.

Because the symmetry clauses are sound, this entails non-colorability of the graph; it is not a DRAT proof of the naive CNF, which is exactly the intractable object the break avoids.

6 Two methodological findings

The program paid for the following two facts in compute; we state them as measured results because we expect them to transfer to any unit-distance colorability effort.

6.1 A portfolio law near the phase boundary

While probing whether a 510-vertex in-class UDG, augmented with several hundred codegree-safe edges, becomes non-5-colorable, we hit a striking solver disagreement on a single fixed instance (a 470-edge augmentation, DIMACS persisted):

CaDiCaL did not decide it in 12+ hours, while MapleChrono returned SAT in 155 seconds — a heuristic-dependent swing of more than $270\times$ on the *identical* CNF.

This is the heavy-tailed runtime behavior expected of k -colorability instances sitting at a satisfiability phase boundary, where the verdict is decided by a handful of branching choices and different heuristics land on wildly different trees. It retroactively explains an earlier instance abandoned as “SAT-intractable” after 66 CaDiCaL-minutes: it was very likely tractable for a different solver and was never re-tried. The operational law:

Portfolio law. Near-threshold k -colorability instances in the unit-distance program must be run as a *solver portfolio* (CaDiCaL for DRAT certification; MapleChrono / Glucose for discovery), never a single solver. A single-solver timeout is not evidence of hardness.

6.2 The regime boundary: structured UNSAT versus phase-transition hardness

It is tempting to read Section 5 as “symmetry breaking makes these instances easy.” That is true only for *structured* UNSAT: sparse, clearly-unsatisfiable graphs (the lineage, the Mycielski tower, de Grey’s graph), where the break removes the dominant redundancy and a learning solver finishes fast. It does *not* tame *phase-transition* hardness. Concretely, the “overshoot” instances — P_{510} plus ~ 1000 codegree-safe and K_4 -safe edges at $k = 5$, deliberately pushed onto the 5-SAT / 5-UNSAT boundary — are a money pit for *both* encodings: naive CaDiCaL did not finish in ~ 100 minutes, and the symmetry-broken encoding hit its conflict budget at 500k / 2M / 10M conflicts (29 s / 157 s / 1387 s, undecided).

The consequence matters for the larger program. Symmetry breaking certifies everything currently *known* about $\chi(\mathbb{R}^2) \geq 5$, but it does *not* unblock the search for a *new* small $\chi \geq 6$ object: that obstruction is intrinsic to the colorability landscape, not an artifact of the encoding. Distinguishing “structured UNSAT” from “phase-transition hardness” is the practical lesson; conflating them (as we briefly did) produces false optimism about what a better encoding can buy.

Section 6.3 qualifies this in the direction that matters. The obstruction is indeed not an artifact of the encoding, and a lever that is not an encoding change does cross it at small orders. It then stops anyway, which is the more informative fact.

6.3 Moving the quantifier inside the solver

Both findings above concern how a *single* colorability question is posed. The program’s harder questions have another shape. Asking whether a class $\mathcal{C}$ contains a member with $\chi \geq k$ is a two-quantifier question, $\exists G \in \mathcal{C} \forall c$, and the usual response collapses the existential half by brute force: generate every member of the class, test each one. The cost is then governed by the *size of the class* rather than by the difficulty of any individual instance, which is why questions of this shape become hopeless long before the colorability tests inside them do.

A third lever applies, and it is neither a better encoding nor a faster solver. SAT modulo symmetries [7] already owns the existential half: a minimality propagator inside the CDCL loop rejects every labeling but the lexicographic representative of each isomorphism class, so the search generates the class up to isomorphism as it goes. A chromatic-number propagator puts the universal half in the same place. On a fully defined candidate it looks for a $(k - 1)$ -coloring; on finding one it learns that at least one currently monochromatic non-adjacent pair must become an edge, and hands the conflict back to CDCL. The $\forall$ -half becomes clause learning inside the same solver that is deciding the $\exists$ -half, and not one member of the class is ever emitted.

On the class this program cares about, the K_4 -free and $K_{2,3}$ -free graphs that every planar unit-distance graph must inhabit, the effect is not incremental. Measured against this repository’s own enumeration cost for the identical question: order 15 falls from 420 seconds to 3.6; order 16 from 66.3 cpu-hours to 74.3 seconds, a factor near 3,200; and order 17, which enumeration measured at more than 80 cpu-days, closes in 0.83 cpu-hours (2,975 core-seconds across seven edge-count cells, 1,202 seconds of wall time on seven cores). Every verdict is negative, and the negative verdict is precisely the one that enumeration makes expensive.

Three things the collapse does not give. First, the answers are not proof-carrying. Writing the same question natively as a 2-QBF, so that the universal block holds a color assignment and the matrix asserts class membership, reproduces unsatisfiable on every nonempty order-17 cell in 45 to 579 seconds by a route sharing neither propagator with the first; but reproducing a verdict is not certifying it, and no machine-checkable object is extracted from either. This is a real gap against the DRAT proofs of Section 5: the propagators are trusted code, and what they buy in speed they cost in checkability.

Second, an unsatisfiable answer is uninformative if the cell is empty for reasons unrelated to coloring, so vacuity has to be controlled rather than assumed. Of the seven order-17 cells, five contain members and two are empty; the negative result is carried entirely by the five. The control is cheap, it is easy to forget, and forgetting it silently converts a theorem into a tautology about nothing.

Third, the lever stops. Order 18 does not close, and it does not close for structurally different attacks alike, which locates the obstruction in the problem at that order rather than in any one formulation. That is the honest shape of the result: a genuine wall was crossed at small orders by moving a quantifier, and the next wall did not move.

7 Reproducibility and honest limitations

Reproducibility. Everything above runs on a single workstation with open-source components: the one-hot and symmetry-broken encoders, the structure- first solver, and the PySAT [6] interface to CaDiCaL [2], Glucose [1], and MapleChrono. The graphs are the published de Grey [4] and Polymath16 [10, 9] UDGs and the standard Mycielskians of C_5 [8]. UNSAT solves emit DRAT proofs [13] on request.

Limitations.

1. *The symmetry break is standard.* Clique-fixing and lex-leader color precedence are textbook [3, 12]. We claim no novelty in the construction, only in its application to these instances and the measurement that it crosses these walls.
2. *Timings are single isolated runs*, not averaged benchmarks; the portfolio law of Section 6.1 is precisely why per-instance single-solver times must be read with care. The qualitative gap (wall versus seconds) is robust; the exact constants are not claimed to be.
3. *No bound on $\chi(\mathbb{R}^2)$ moved.* This is tooling and certification, not a new mathematical result. The regime boundary of Section 6.2 is explicit that the search for $\chi \geq 6$ is not advanced by faster decision procedures.
4. *The structure-first solver is a research instrument*, correct and transparent, not a competitor to CDCL at scale; its value is as the source of the exported break and as a pattern counter.

8 Conclusion

The lower-bound side of the Hadwiger-Nelson problem is a stack of k -colorability certificates, and the naive one-hot encoding handicaps every one of them with the $k!$ color symmetry. Exporting a sound symmetry break to a production CDCL solver removes the handicap and makes the *entire* known $\chi(\mathbb{R}^2) \geq 5$ program self-certifiable on a single workstation, with machine-checkable DRAT proofs, including instances previously reported intractable. The same exercise yields three transferable lessons: run a portfolio near the phase boundary; do not mistake structured-UNSAT speed for progress on phase-transition hardness; and when the question is really $\exists G \forall c$, consider moving the inner quantifier into the solver as a propagator rather than paying for the class by enumerating it. The third lever is the strongest of the three and still runs out: it settles the class question through order 17 at a cost enumeration could not approach, and order 18 resists it as firmly as it resists everything else. The break certifies what is known, and the collapse cheapens what can be checked; finding the next bound remains a mathematical problem, not a solver-engineering one.

Acknowledgments

This note arose from an ongoing computational program on the Hadwiger-Nelson problem. The graphs studied derive from the de Grey [4] and Polymath16 [10] constructions. All computation was carried out on a single workstation using open-source SAT solvers (CaDiCaL, MapleChrono, Glucose) through the PySAT toolkit, with the pure-Python components run under PyPy. No external funding supported this work.

References

- [1] Gilles Audemard and Laurent Simon. Predicting learnt clauses quality in modern SAT solvers. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI 2009)*, pages 399–404, 2009. Glucose solver.
- [2] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020. In Tomas Balyo,

- Nils Froleyks, Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda, editors, *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, volume B-2020-1 of *Department of Computer Science Report Series B*, pages 50–53. University of Helsinki, 2020. CaDiCaL solver description.
- [3] James Crawford, Matthew Ginsberg, Eugene Luks, and Amitabha Roy. Symmetry-breaking predicates for search problems. In *Principles of Knowledge Representation and Reasoning (KR’96)*, pages 148–159. Morgan Kaufmann, 1996. Lex-leader symmetry-breaking predicates.
 - [4] Aubrey D. N. J. de Grey. The chromatic number of the plane is at least 5. *Geombinatorics*, 28(1):18–31, 2018. Preprint: arXiv:1804.02385.
 - [5] Marijn J. H. Heule. Computing small unit-distance graphs with chromatic number 5. *Geombinatorics*, 28(1):32–50, 2018. SAT verification and reduction of de Grey’s graph. Preprint: arXiv:1805.12181 [math.CO].
 - [6] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In Olaf Beyersdorff and Christoph M. Wintersteiger, editors, *Theory and Applications of Satisfiability Testing (SAT 2018)*, volume 10929 of *LNCS*, pages 428–437. Springer, 2018.
 - [7] Markus Kirchweger and Stefan Szeider. SAT modulo symmetries for graph generation. In *27th International Conference on Principles and Practice of Constraint Programming (CP 2021)*, volume 210 of *LIPICs*, pages 34:1–34:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. Isomorphism handled by a lexicographic-minimality propagator inside the CDCL solver.
 - [8] Jan Mycielski. Sur le coloriage des graphes. *Colloquium Mathematicum*, 3:161–162, 1955.
 - [9] Jaan Parts. Graph minimization, focusing on the example of 5-chromatic unit-distance graphs in the plane. *Geombinatorics*, 29(3):137–166, 2020.
 - [10] Polymath, D. H. J. Polymath16: The chromatic number of the plane. Online collaborative project, Polymath wiki and blog of Dustin G. Mixon and Aubrey de Grey, 2018. Reduced unit-distance graphs with chromatic number at least 5 to roughly 510 vertices.
 - [11] Patrick Prosser. Hybrid algorithms for the constraint satisfaction problem. *Computational Intelligence*, 9(3):268–299, 1993. Conflict-directed backjumping (CBJ).
 - [12] Karem A. Sakallah. Symmetry and satisfiability. In Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors, *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*, pages 289–338. IOS Press, 2009. Survey of symmetry breaking in SAT; lex-leader predicates.
 - [13] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing (SAT 2014)*, volume 8561 of *LNCS*, pages 422–429. Springer, 2014.